

CSC1031: OOP

Week 4

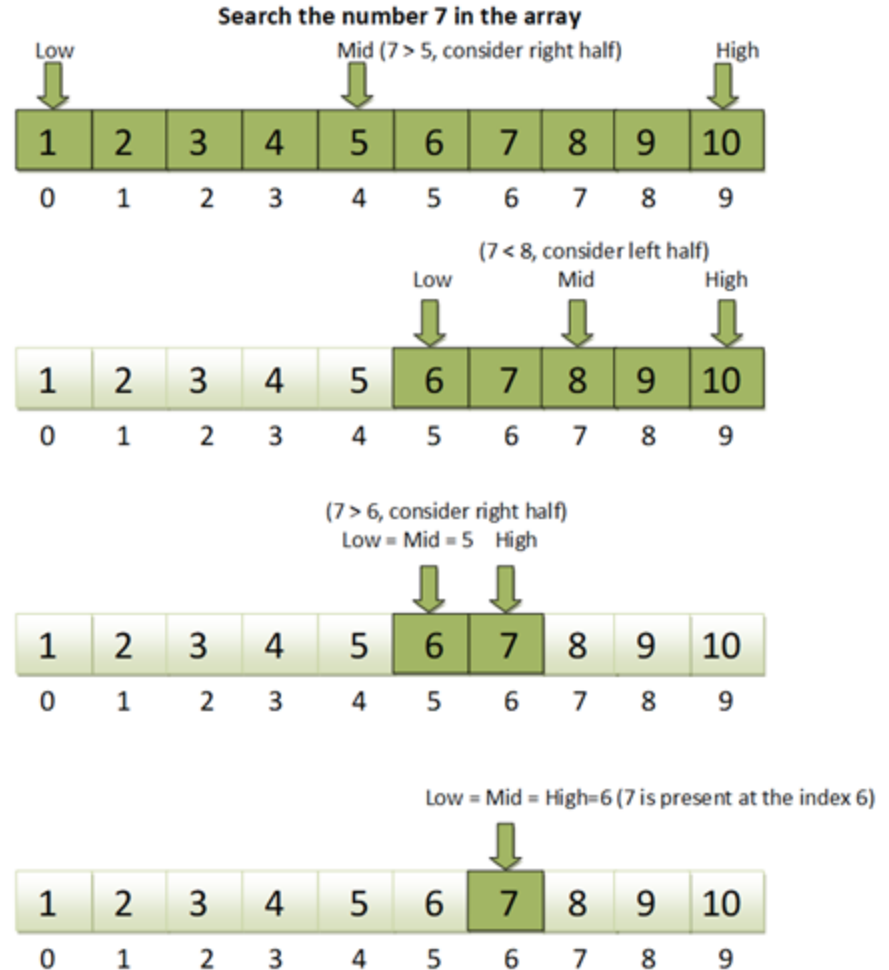
Inheritance, overriding

Binary search

Binary Search is an **efficient searching algorithm** used to find an element in a **sorted array**.

It follows a **divide and conquer** approach:

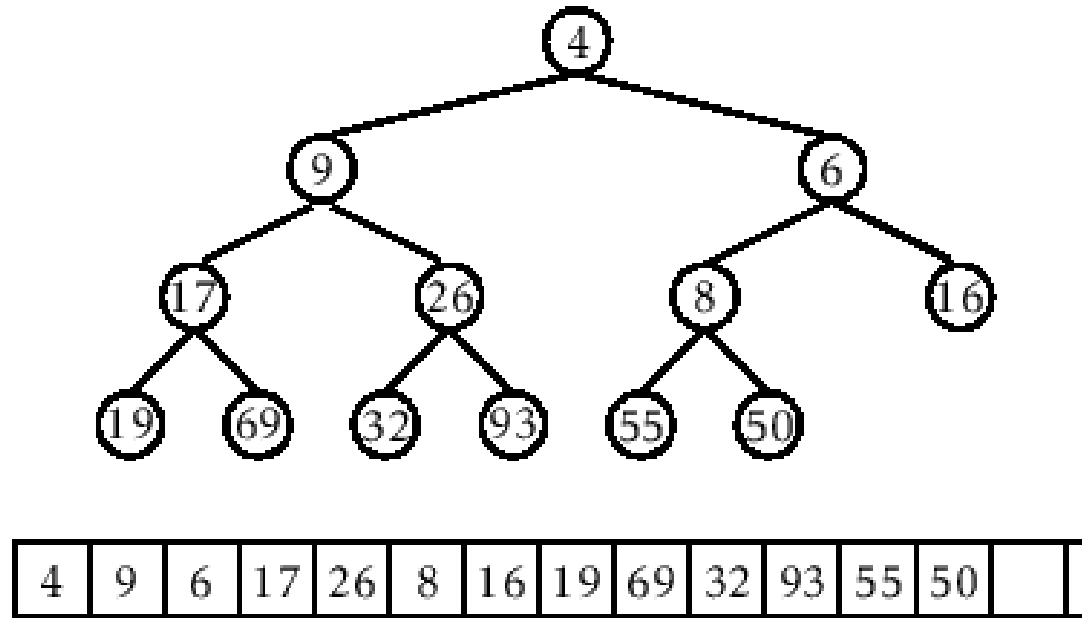
1. Find the **middle element** of the array.
2. If the target is **equal** to the middle element → **Found!**
3. If the target is **smaller**, search the **left half**.
4. If the target is **greater**, search the **right half**.
5. Repeat until the element is found or the search space is empty.



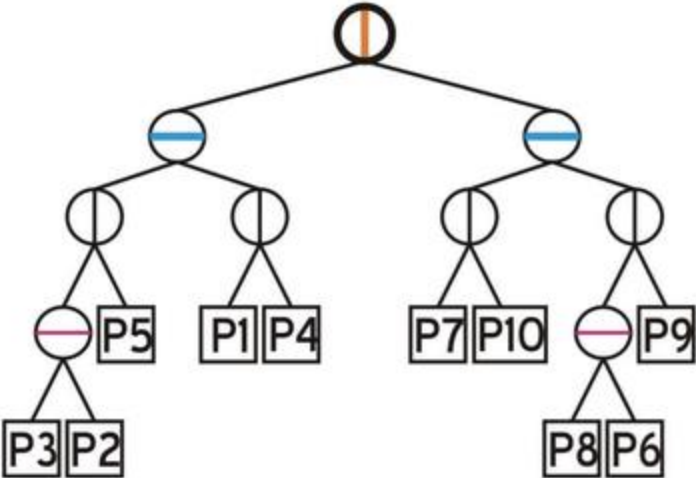
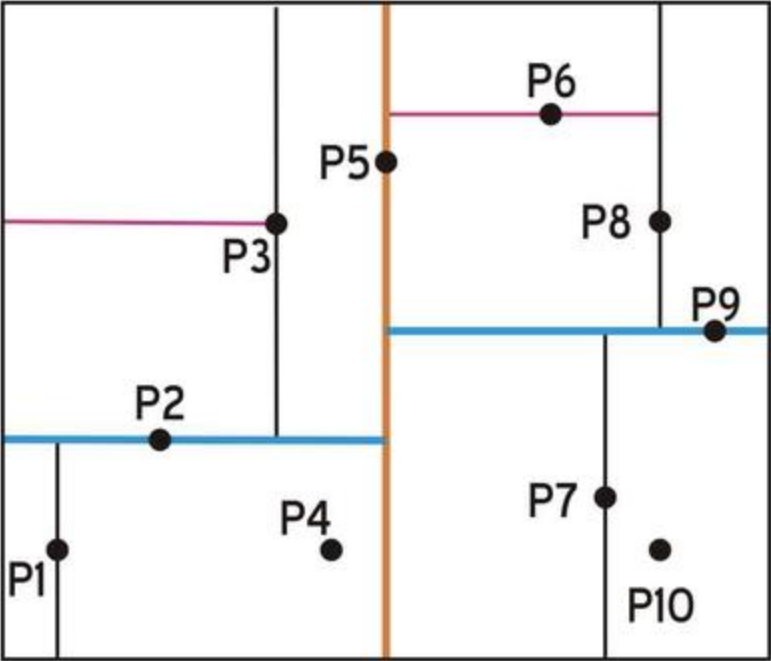
Binary Tree

A **Binary Tree** is a hierarchical data structure in which:

- Each **node** has **at most two children** (left and right).
- The **topmost node** is called the **root**.
- The last-level nodes with no children are called **leaf nodes**.



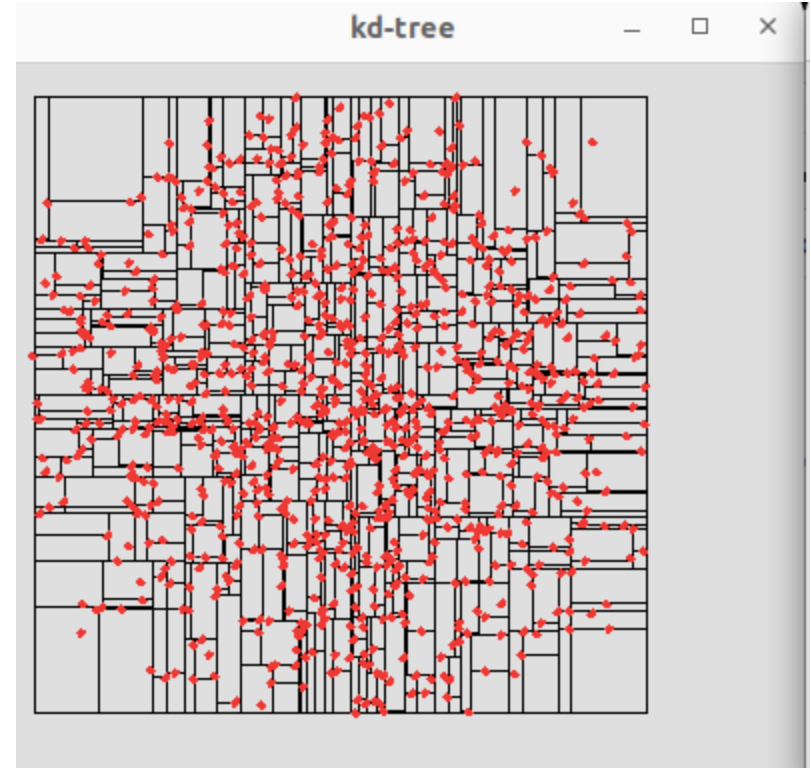
KD-tree (K-Dimensional Tree)



KD-tree (K-Dimensional Tree)

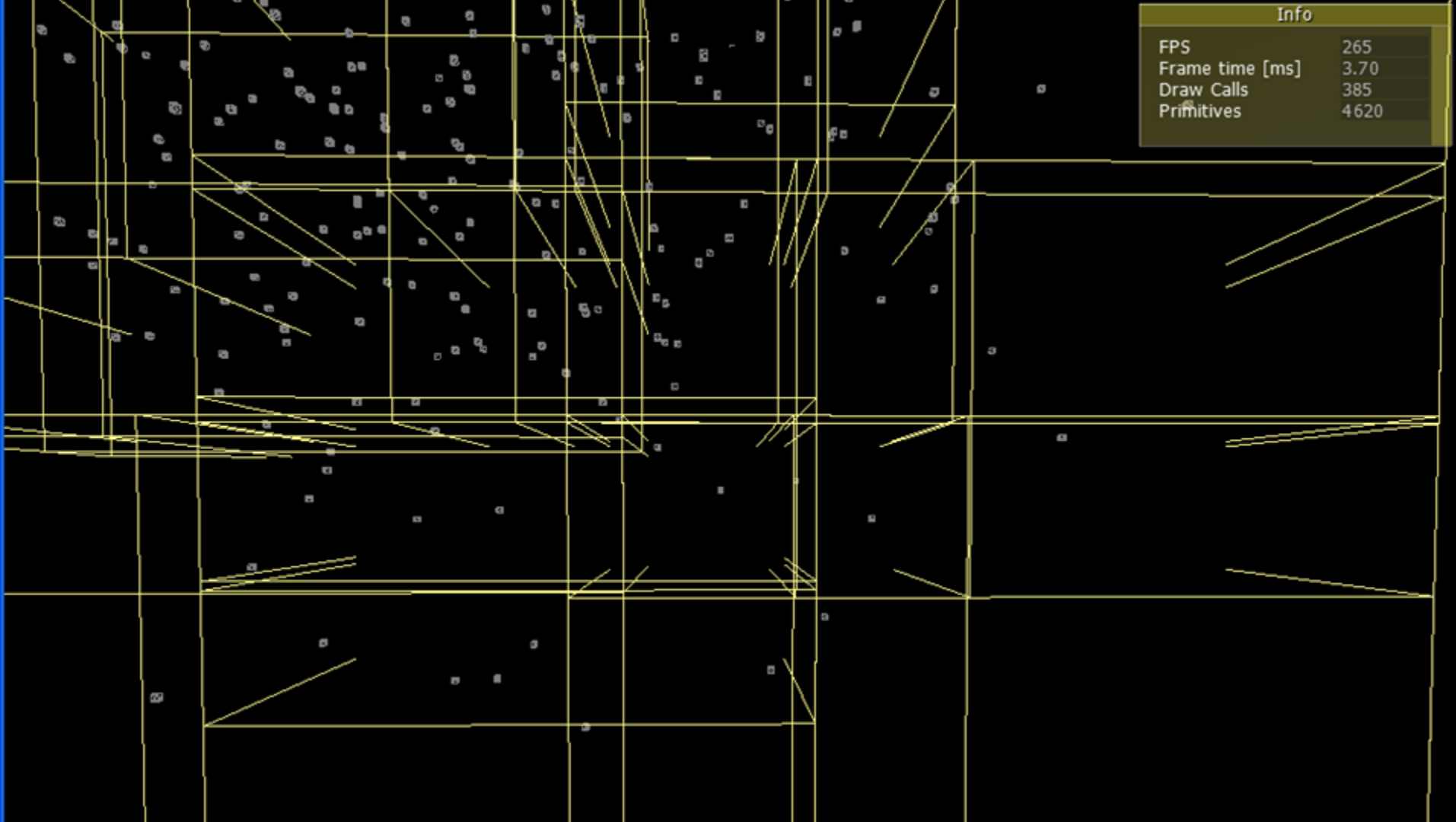
A KD Tree (k-dimensional tree) is a **space-partitioning data structure** used for **efficient searching in multi-dimensional spaces**.

- It recursively **divides the space** into regions by selecting a **pivot** at each level
- Mainly used for **nearest neighbor search, range search, and clustering**



Info

FPS	265
Frame time [ms]	3.70
Draw Calls	385
Primitives	4620



Compilation Errors vs. Runtime Errors in Java

Compilation Errors

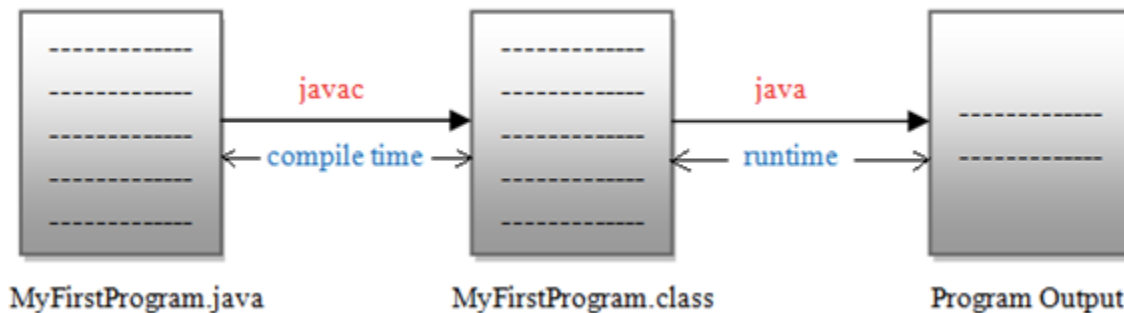
- **Detected at compile time** (before running the program).
- Occur due to **syntax mistakes**, incorrect **data types**, missing semicolons, undeclared variables, etc.

```
int x = "hello"; // ✗ Type mismatch
```

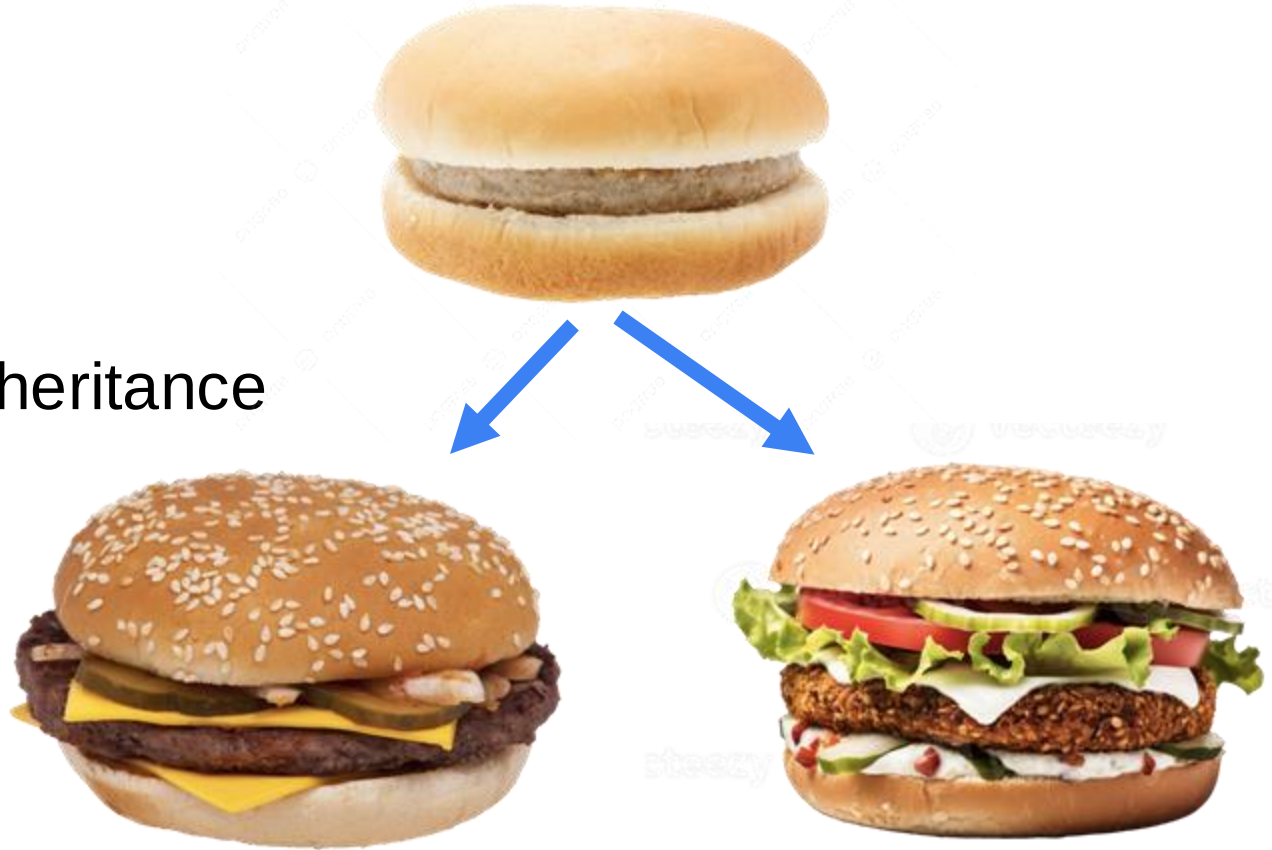
Runtime Errors

- **Occur while the program is running**, even if it compiled successfully.
- Typically caused by **invalid operations**, such as dividing by zero, accessing a null reference, or exceeding array bounds.

```
int a = 10 / 0; // ✗ ArithmeticException
```



Introduction to Inheritance



Definition

- Inheritance allows one class to **acquire properties and methods** of another class
- Enables **code reuse** and **extensibility**.

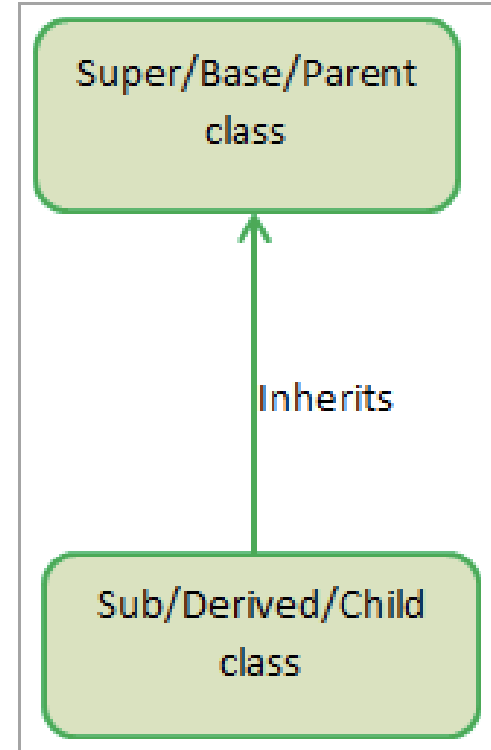
```
class Parent {  
    int x = 10;  
}  
  
class Child extends Parent {  
    // Child inherits Parent  
    int y = 20;  
}
```

Types of Inheritance in Java

- **Single Inheritance** – One class inherits another
- **Multilevel Inheritance** – A class inherits from a class that is already inherited
- **Hierarchical Inheritance** – Multiple classes inherit from a single parent.
- Java does not support **multiple inheritance** with classes

What is Single Inheritance?

- **Single Inheritance** is a mechanism where a **single child class inherits from a single parent class**
- The child class **gains access to the fields and methods** of the parent class, allowing **code reuse and extension**
- It promotes **modularity** and reduces **redundancy** in code.



Syntax of Single Inheritance

```
class Parent {  
    void show() { System.out.println("This is the Parent class"); }  
}  
  
class Child extends Parent {  
    void display() { System.out.println("This is the Child class"); }  
}
```

Syntax of Single Inheritance

The **extends** keyword is used to inherit from a class

```
class Parent {  
    void show() { System.out.println("This is the Parent class"); }  
}  
  
class Child extends Parent {  
    void display() { System.out.println("This is the Child class"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.show();    // Inherited method  
        obj.display(); // Child class method  
    }  
}
```



The child class gets access to **non-private members** of the parent class

Single Inheritance

Code Reusability – Avoids writing duplicate code.

Improves Maintainability – Changes in the parent class automatically reflect in the child class.

Extensibility – Allows adding new functionality in derived classes without modifying the parent.

Banking Systems:

BankAccount → SavingsAccount

E-commerce Platforms:

User → VIP User

Automobile Industry:

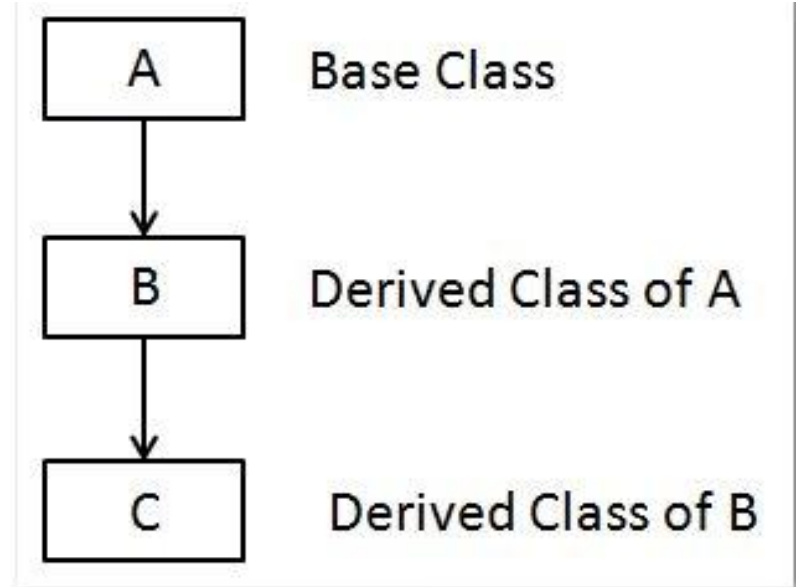
Car → Electric Car

What is Multilevel Inheritance?

Multilevel Inheritance occurs when a class derives from a **child class**, which itself extends another class.

It forms a **chain of inheritance**, where properties and methods are passed down through multiple levels.

Grandparent → Parent → Child



Syntax of Multilevel Inheritance

```
class Grandparent {  
    void grandparentMethod() { System.out.println("Grandparent method"); }  
}
```

Syntax of Multilevel Inheritance

```
class Grandparent {  
    void grandparentMethod() { System.out.println("Grandparent method"); }  
}  
  
class Parent extends Grandparent {  
    void parentMethod() { System.out.println("Parent method"); }  
}
```

Syntax of Multilevel Inheritance

```
class Grandparent {  
    void grandparentMethod() { System.out.println("Grandparent method"); }  
}  
  
class Parent extends Grandparent {  
    void parentMethod() { System.out.println("Parent method"); }  
}  
  
class Child extends Parent {  
    void childMethod() { System.out.println("Child method"); }  
}
```

Syntax of Multilevel Inheritance

```
class Grandparent {  
    void grandparentMethod() { System.out.println("Grandparent method"); }  
}  
  
class Parent extends Grandparent {  
    void parentMethod() { System.out.println("Parent method"); }  
}  
  
class Child extends Parent {  
    void childMethod() { System.out.println("Child method"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.grandparentMethod(); // Inherited from Grandparent  
        obj.parentMethod();      // Inherited from Parent  
        obj.childMethod();       // Defined in Child  
    }  
}
```

Examples

Banking System

BankAccount → SavingsAccount → PremiumSavingsAccount

Automobile Industry

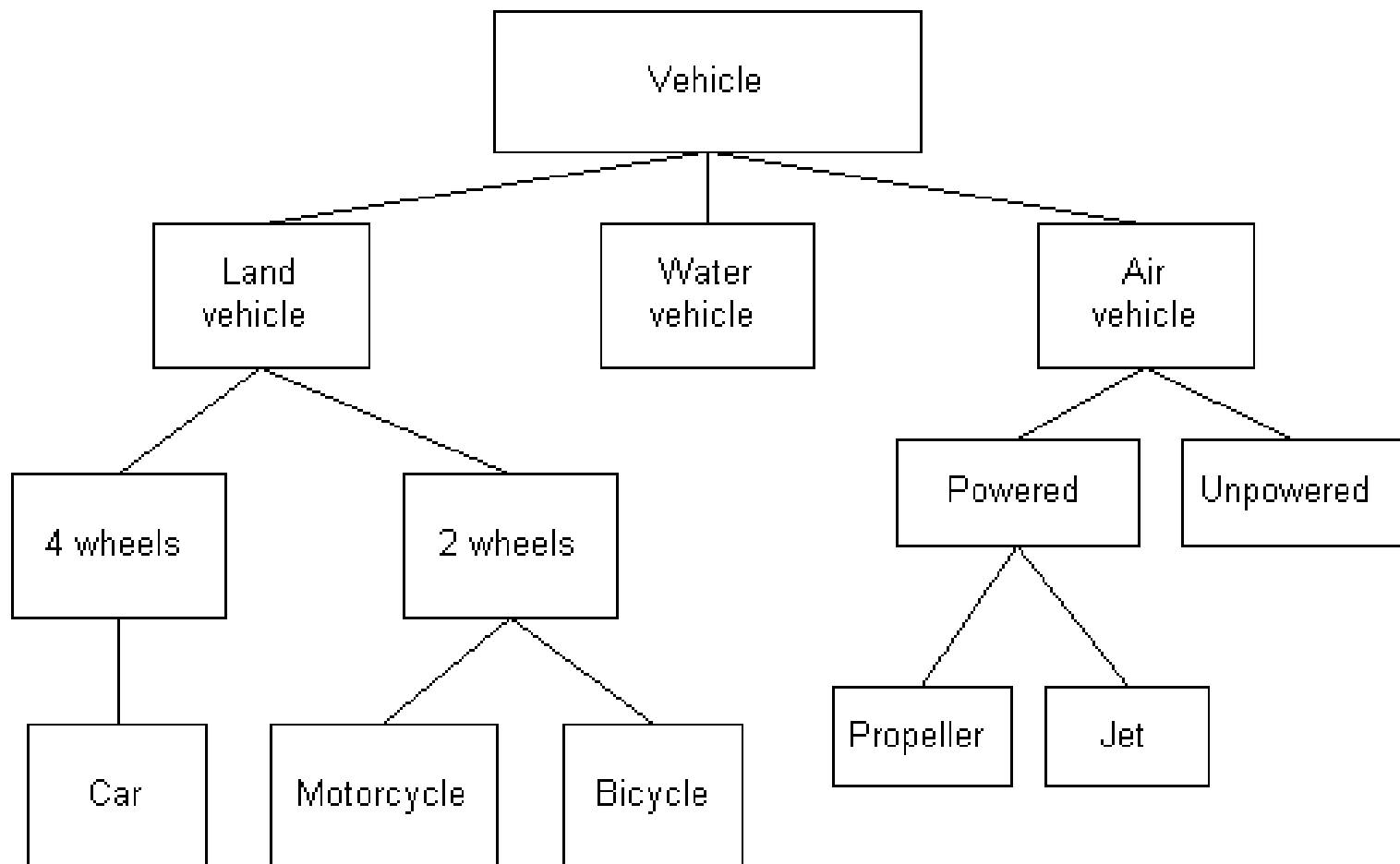
Vehicle → Car → ElectricCar

Education System

Person → Student → GraduateStudent

Employee Hierarchy

Employee → Manager → SeniorManager



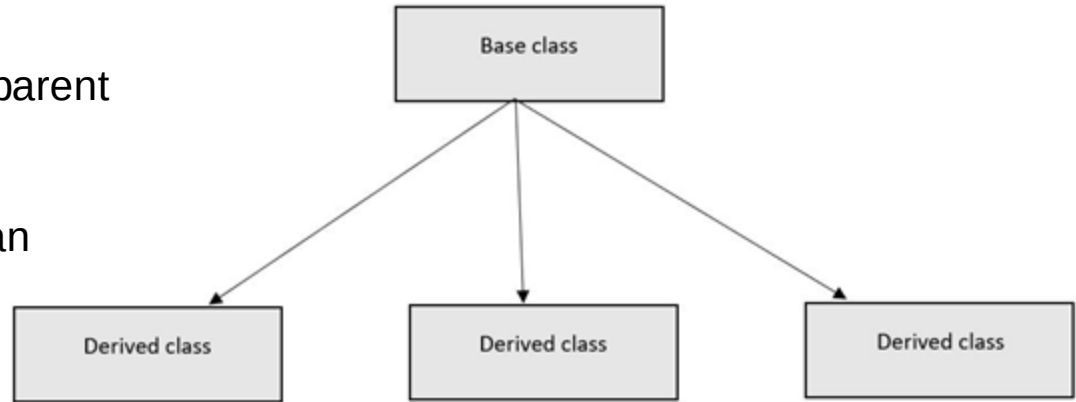
What is Hierarchical Inheritance?

Hierarchical Inheritance occurs when **multiple child classes inherit from a single parent class**.

Each child class **inherits common properties and methods** from the parent while defining its own behavior.

It allows code reuse and promotes an organized structure.

One Parent → Multiple Children



Syntax of Hierarchical Inheritance

```
class Parent {  
    void commonMethod() { System.out.println("Common method from Parent class"); }  
}
```

```
class Child1 extends Parent {  
    void child1Method() { System.out.println("Child1-specific method"); }  
}
```

```
class Child2 extends Parent {  
    void child2Method() { System.out.println("Child2-specific method"); }  
}
```

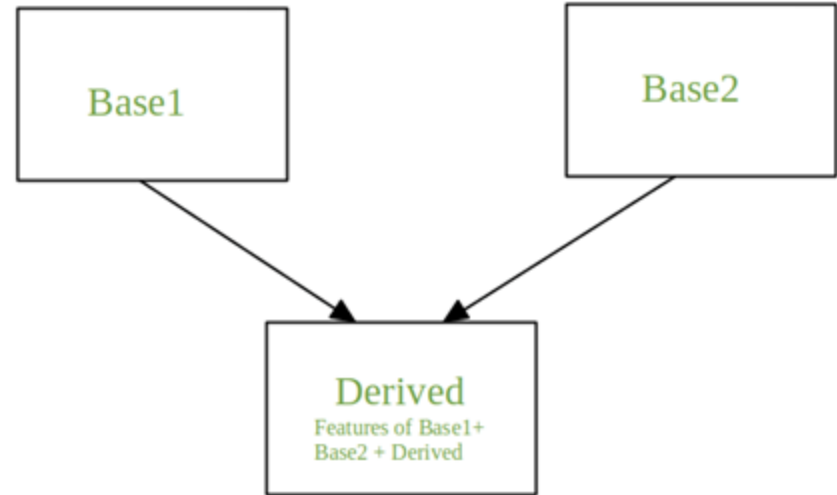
```
public class Main {  
    public static void main(String[] args) {  
        Child1 obj1 = new Child1();  
        obj1.commonMethod(); // Inherited from Parent  
        obj1.child1Method(); // Child1 method  
  
        Child2 obj2 = new Child2();  
        obj2.commonMethod(); // Inherited from Parent  
        obj2.child2Method(); // Child2 method  
    }  
}
```

What is Multiple Inheritance?

Multiple Inheritance is when a **child class inherits from two or more parent classes**.

Java **does NOT support multiple inheritance with classes** to prevent **ambiguity issues** (Diamond Problem).

However, Java **supports multiple inheritance using interfaces**.

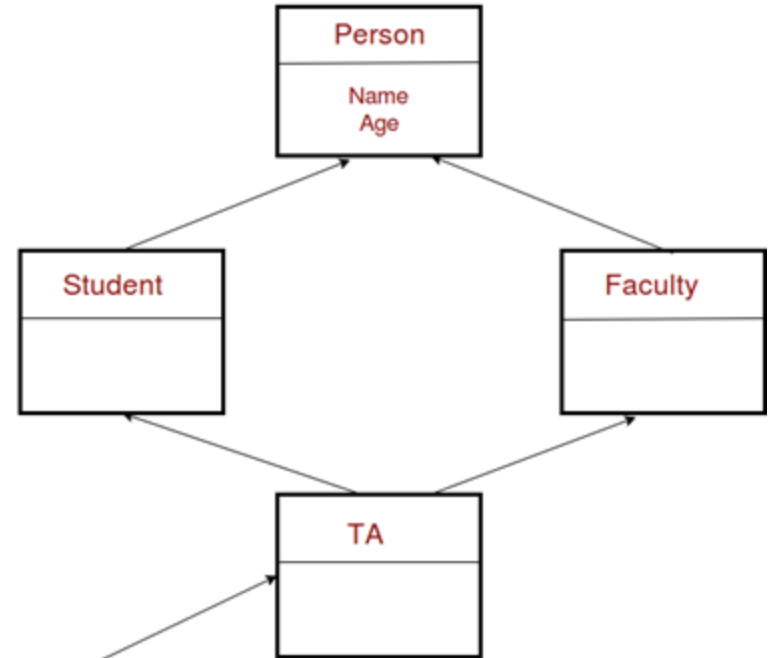


Why Java Does Not Support Multiple Inheritance?

Ambiguity Problem

(Diamond Problem)

- If two parent classes have the same method and the child class inherits from both, **which method should be called?**
- This leads to **compiler confusion** and possible runtime errors.



Name and Age needed only once

Applications of Multiple Inheritance

E-Commerce System: User Roles

- `Buyer` and `Seller` → `User` implements both.

Automobile Industry: Hybrid Vehicles

- `ElectricVehicle` and `FuelVehicle` → `HybridCar` implements both.

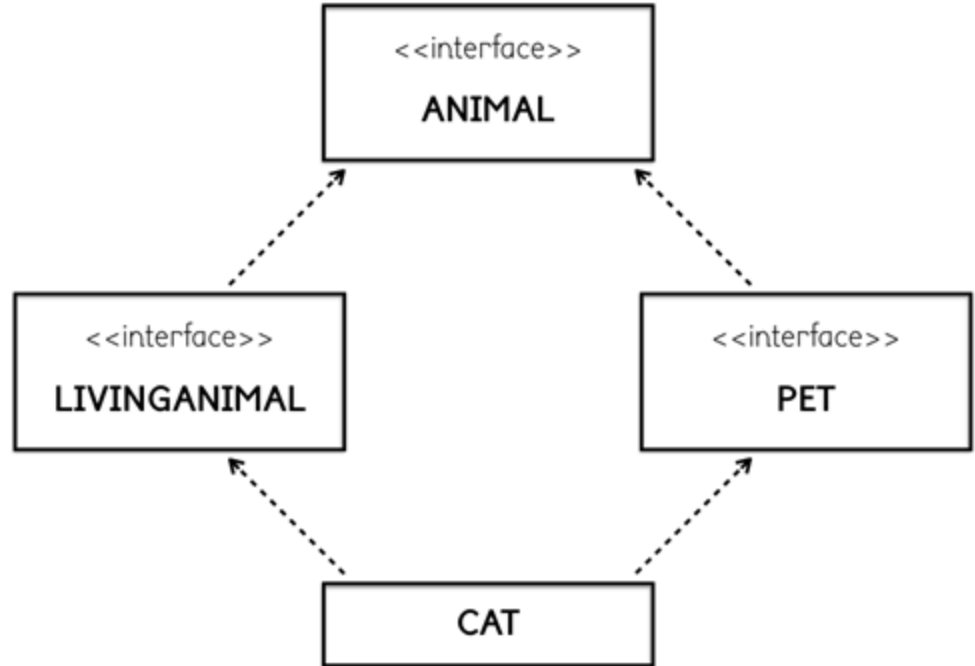
Banking System: Multiple Account Features

- `LoanAccount` and `SavingsAccount` → `UserAccount` implements both.

What is an Interface in Java?

An **interface** in Java is a blueprint of a class that contains **only abstract methods** and **constant variables**.

- **Defines a contract:** Any class that **implements** an interface must provide implementations for its methods.
- **Supports multiple inheritance:** A class can implement **multiple interfaces**, overcoming Java's single inheritance limitation.
- **Enhances abstraction:** Interfaces help **define behavior** without specifying how it should be implemented.



Multiple Inheritance with Interfaces

```
interface Animal {  
    void eat();  
}  
  
interface Walkable {  
    void walk();  
}  
  
// A class implementing multiple interfaces  
class Dog implements Animal, Walkable {  
    public void eat() {  
        System.out.println("Dog is eating");  
    }  
  
    public void walk() {  
        System.out.println("Dog is walking");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        myDog.eat(); // Output: Dog is eating  
        myDog.walk(); // Output: Dog is walking  
    }  
}
```

Common Mistakes & Best Practices

Deep Inheritance Chains – Too many levels make code **hard to maintain**.

Constructor Calls – Always use `super()` when needed for proper initialization.

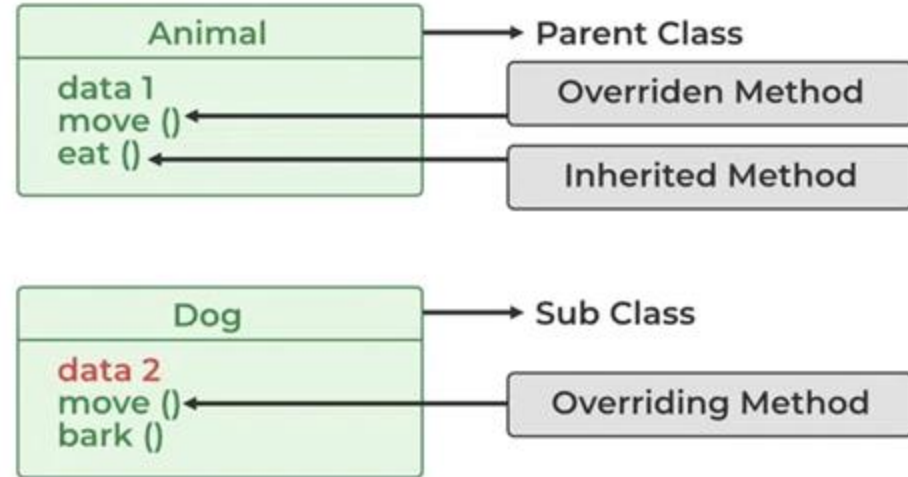
Avoid Diamond Problem – Java does **not** support multiple inheritance with classes to prevent this.

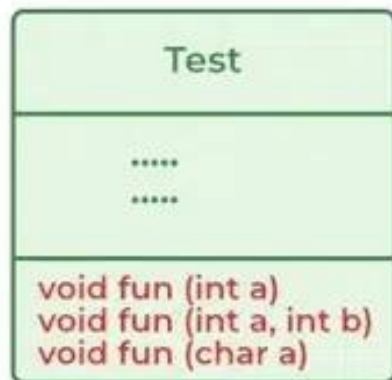
What is Method Overriding?

Method Overriding allows a **child class** to **provide a specific implementation** of a method that is already defined in its **parent class**.

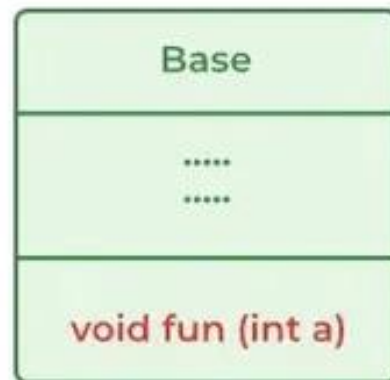
The method in the child class **must have the same name, return type, and parameters** as in the parent class.

Overriding is used for **runtime polymorphism** and enables **dynamic method dispatch**.





Overloading



Overriding

Syntax of Method Overriding

```
class Parent {  
    void show() {  
        System.out.println("This is the Parent class");  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void show() { // Overriding the method  
        System.out.println("This is the Child class");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Parent obj = new Child(); // Upcasting  
        obj.show(); // Calls overridden method in Child class  
    }  
}
```

Rules for Method Overriding

Method signature (name, parameters) must be the same.

The overridden method cannot be more restrictive in access modifier:

- `public` → Must stay `public`
- `protected` → Can be `protected` or `public`, but not `private`
- `private` methods **cannot** be overridden.

Final methods cannot be overridden (methods marked as `final`).

Static methods cannot be overridden, but can be re-declared in the child class.

Access Modifiers

```
class Parent {  
    private int privateVar = 10;    //  Not inherited  
    int defaultVar = 20;            //  Inherited (same package only)  
    protected int protectedVar = 30; //  Inherited (even in subclass outside package)  
    public int publicVar = 40;       //  Always inherited  
}  
  
class Child extends Parent {  
    void display() {  
        // System.out.println(privateVar); //  Error: private not accessible  
        System.out.println(defaultVar);    //  Accessible (same package)  
        System.out.println(protectedVar);  //  Accessible  
        System.out.println(publicVar);     //  Accessible  
    }  
}
```

Using super in Inheritance

Calls the **parent class method** inside the child class.

Helps in **method overriding** and constructor chaining

```
class Parent {  
    void display() {  
        System.out.println("This is the Parent method");  
    }  
}  
  
class Child extends Parent {  
    void display() {  
        super.display(); // Calls the parent method  
        System.out.println("This is the Child method");  
    }  
}
```

Best Practices for Inheritance & Access Modifiers

- Use `protected` carefully – Only when subclasses need access.
- Keep fields `private` and provide getters/setters for encapsulation.
- Use `super` to call parent methods in overridden methods.
- Avoid deep inheritance chains – They make debugging difficult.
- Favor composition over inheritance when possible.